Dynamics On Complex Networks

Fergal Brennan

Kevin O'Reilly

AM3064/AM6015 Written Report

School of Mathematical Sciences

University College Cork

Ireland

March 2026

This is to certify that the work I am submitting is my own and has not been submitted for another degree, either at University College Cork or elsewhere. All external references and sources are clearly acknowledged and identified within the contents. I have read and understood the regulations of University College Cork concerning plagiarism.

Fergal Brennan

Date: 27/03/2026

Signature: Fergal Brennan

Kevin O'Reilly

Date: 27/03/2026

Signature: Kevin O'Reilly

Author contributions

Fergal Brennan wrote chapters 2, 3.1, 4.4 and 5.4, 6 and the code in A.1. Kevin O'Reilly wrote the abstract, chapters 1, 3.2-3.5, 4.1-4.3, 5.1-5.3 and the code in A.2.

Contents

1 Introduction to Networks **1**

1.1 Motivation 1

1.2 Basic Network Properties 1

1.3 The Graph Laplacian 2

2 Models of Networks **3**

2.1 Random Graph 3

2.2 Small World 4

2.3 Scale-Free 6

3 Synchrony and Contagion Spread **8**

3.1 History of Synchrony 8

3.2 Kuramoto Model 9

3.3 Algbebraic Connectivity 10

3.4 Simple vs. Complex Contagion 10

3.5 Research Questions 10

4 Methods **12**

4.1 Network Generation 12

4.2 Kuramoto Model 13

4.3 Threshold Model 13

4.4 SIR Model 14

5 Results **15**

5.1 Network Characteristics 15

5.1.1 Basic Properties 15

5.1.2 Degree Distributions 15
5.1.3 Algebraic Connectivity 17
5.2 Kuramoto Synchronisation 17
5.3 Complex Contagion 19
5.3.1 Threshold Cascades 19
5.3.2 The Role of Clustering 19
5.4 Simple Contagion - SIR 20

6 Conclusion **22**

A Code **23**
A.1 Python-Fergal 23
A.2 Python-Kevin 29

Abstract

This paper describes how network structure influences dynamical processes. Three of the most commonly researched networks models were used, the Erdos-Reyni random graph, the Watts-Strogatz small-world network and the Barabasi-Albert scale-free network. The synchronisation of a system of couple oscillators, the spread of a simple contagion (disease) and the spread of a complex contagion (likened to adopting a new diet) were simulated on each of these networks. The greater algebraic connectivity of the scale-free network relative to the others meant synchronisation and simple contagion spread occurred more easily due to its hub-based topology. The less globally connected and more locally clustered small-world network struggled the most to synchronise and saw slower spread of the simple contagion. However, its clusters provided the architecture for the greatest spread of the complex contagion, which required multiple exposures to spread, by providing local reinforcement in its dense communities.

Chapter 1

Introduction to Networks

1.1 Motivation

Many of the most important real-life systems, naturally occurring or otherwise, can be best understood and analysed not as isolated components, but as a collection of interacting components. A society is a network of individuals who influence each other through the connection of friendship, professional ties, academic relationships etc... A power grid is a network of transformers and transmission lines. Our ecosystems can be mapped as networks of species and predator/prey relationships. In each case, the structure of the networks, who is connected to whom and the nature of the relationship, profoundly shapes the behaviour of the whole system, in a way that cannot be understood by solely examining individual parts.

1.2 Basic Network Properties

Network theory has much overlap with graph theory in its mathematical structure, but is generally used in a more applied context to map some dynamic process. A graph consists of a set V of vertices and E of edges connecting vertices, whereas network theory often uses the terms nodes and links. A link or an edge which connects pairs of nodes can be directed, pointed specifically from one node to another, or can be undirected, simply connecting two nodes. This link may or may not carry a weighting encoding the strength of interaction or relationship, depending on the system being modelled. The complete connectivity pattern can be represented by the adjacency matrix A, where $A_{ij} = 1$ if there

is a connection between node i and node j, and 0 otherwise (for unweighted, undirected graphs). The most elementary property of a node is its degree k. This represents the number of edges attached to it, quantifying how well-connected it is to the rest of the network. The set of degrees across all nodes defines the degree distribution P(k), which is fundamental to understanding a given network's topology. It shows us the probability that a node has exactly k neighbours. As well as degree, several other structural measures prove useful. The local clustering coefficient C_i of a node measures the fraction of its neighbours that are themselves connected to each other, demonstrating cliquishness. For each node i, C_i is defined as:

$$C_i = \frac{2e_i}{k_i(k_i - 1)}$$

where e_i is the number of edges between neighbours of node i, and k_i is the degree of node i. This measures the fraction of possible triangles centered at node i that actually exist. The average path length L is the mean of the shortest paths between all pairs of nodes in the network, quantifying how efficiently information or influence can be propagated across the network. The diameter is the largest of all the shortest paths. Together, these measures illustrate how compact or locally clustered a network is.

1.3 The Graph Laplacian

A key object in the analysis of network dynamics is the graph Laplacian L = D - A where D is the diagonal degree matrix with $D_{ii} = k_i$, and A is the adjacency matrix. The Laplacian is a positive semi-definite matrix whose eigenvalues $0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_N$ encode deep structural information about the network. The second smallest eigenvalue of L, λ_2, which is also the smallest non-zero eigenvalue, is called the algebraic connectivity and it measures how well-connected the network is. This matrix governs how quickly diffusion and synchronisation processes relax to their steady states.

Chapter 2

Models of Networks

There are many different models of networks that are useful for different applications, for example using a regular nearest neighbor model would be useful when trying to emulate the synchronous behaviour of fireflies (Photuris frontalis)[5]. A regular network (all nodes have the same degree) would be appropriate since each firefly only sees its nearest neighbours. Other useful network models include random graphs, small world networks and scale-free networks. We will be focusing on these three.

2.1 Random Graph

The main model used for a random graph was developed by Erdos and Reyni[1]. In this model the number of nodes N and the probability of connecting p are chosen. p determines whether pairs of nodes are connected or not. In this way the probability that the graph produced has M edges goes like

$$p^M(1-p)^{\binom{N}{2}-M}$$

Since there's $\binom{N}{2}$ possible edges to choose from, the degree distribution should follow a binomial distribution. This equation is assuming there are no labelled nodes, so we need to multiply by how many graphs are possible with each M edges. Hence,

$$P(M,p,N) = \binom{\binom{N}{2}}{M}\left(p^M(1-p)^{\binom{N}{2}-M}\right)$$

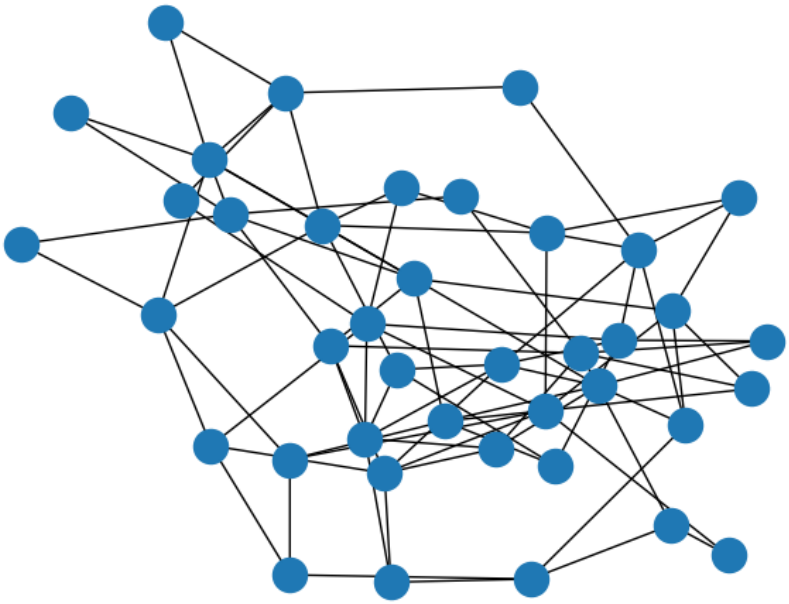
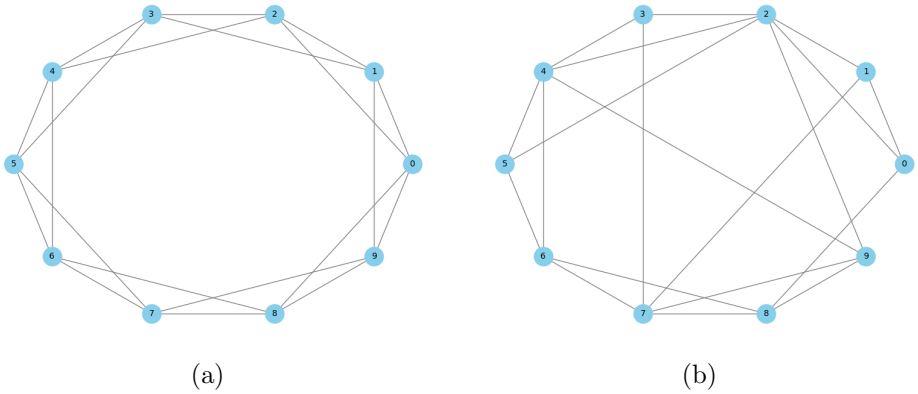


Figure 2.1: Erdos Reyni Random Graph from Networkx using Python. Parameters chosen were $N = 40$, $p = 0.1$ and 21st seed.

gives the probability that some graph with these parameters exists. This implies we can vary any of these three parameters, which is true. However the most widely used way to obtain a random graph is fixing N and p and letting M vary. Notice again this is a binomial distribution which can be approximated as a Poisson distribution. The graph in Figure 2.1 uses Networkx[12] in Python using an Erdos Reyni function, which takes a number of nodes input N and a probability of edge input p to produce a random graph.

2.2 Small World

In the 1950s mathematician Manfred Kochen and political scientist Ithiel de Sola Pool collaborated on a mathematical manuscript, which described the mechanics of social networks (not published until 1978)[7]. Stanley Milgram was visiting at the time and collaborated on this research. The paper itself brought about lots of unanswered questions, one of which was the degree of connectedness in actual social networks[6]. So, in 1967 Stanley Milgram was the first to study small world networks. In the study participants were chosen from Omaha, Nebraska and Wichita, Kansas and their task was to find their given person in Boston Massachusetts. These cities are distant both socially and geographically. Letters were sent out to willing participants containing basic information



(a) (b)

Figure 2.2: Using rewiring probability $p = 0.3$ on a regular network of $N = 10$ nodes each of degree $k = 4$. **(a)** being our regular graph and **(b)** being our small world graph. We can see the community structure and the hubs remain, but there are now shortcuts in **(b)** because of the rewiring.

about the target person. If the receiver did not know the person then they were told to send the letter to someone they know, that they think might know them. Once the next person receives the information the process repeats until the letter reaches the target individual. The amount of people it took to get to the target person is the degree of connectedness. The study found that, on average, the people who took part were about 6 degrees seperated from the final individual[6].

Hungarian author Frigyes Karinthy, first suggested this 6 degrees concept back in 1929 in his short story "Chains"[8].

Watts and Strogatz introduced a mathematical model in 1998 that formalized small-world networks as structures lying between regular lattices and random graphs, exhibiting both high clustering and short path lengths[9]. Since regular graphs have high clustering and random graphs usually have short path lengths, Watts and Strogatz hypothesised that the small world graph (which was known to have these properties) would lie somewhere in-between the two. Their method involved taking a regular graph with N nodes with degree k each. Then by using probability p to rewire edges at random, a graph with the same kind of community structure (high clustering) but with shorter path lengths is obtained. Hence a small world graph is obtained. This method can be seen in Figure 2.2. It's found that if the rewiring probability p is any higher than ≈ 0.5 the graph behaves like a random graph and has a relatively low global clustering coefficient. So in our experiments we keep our probability p well below this.

2.3 Scale-Free

Scale-free networks, first developed by Laszlo Barabasi and Reka Albert to describe networks with a strange degree distribution [2]. The kind of networks found in the paper had a degree distribution which followed the power law,

$$P(k) = k^{-\gamma} \tag{2.1}$$

so the probability $P(k)$ that a node in the network had a degree k follows equation (2.1) where $\gamma \in \mathbb{R}$. This model of network was developed to describe networks that have special nodes that have a much larger degree than most of the other nodes. Taking the World Wide Web (WWW) as an example where the nodes are the documents and the edges are the links, linking one document to another. It was found that there are just a few documents with most of links (edges) while most documents have not many links at all. This preferential attachment exists because some documents are popular, while most are not. To imitate this preferential attachment in their model, Barabasi and Albert used another probability distribution

$$\Pi_i = \frac{k_i}{\sum_{j=1}^{N(n)} k_j}$$

where Π_i is the probability that the nth added node will be attached to the ith node in the system and k_j is the degree of node j and the number of nodes N is a function of the number of nodes added in, n. This probability is skewed preferentially to nodes with higher degrees. So in turn when the final graph is complete we see a degree distribution which follows the power law, as a result of this preferential attachment. This network can be seen in Figure 2.3 where the lighter colour represents high degree and darker, lower degree.

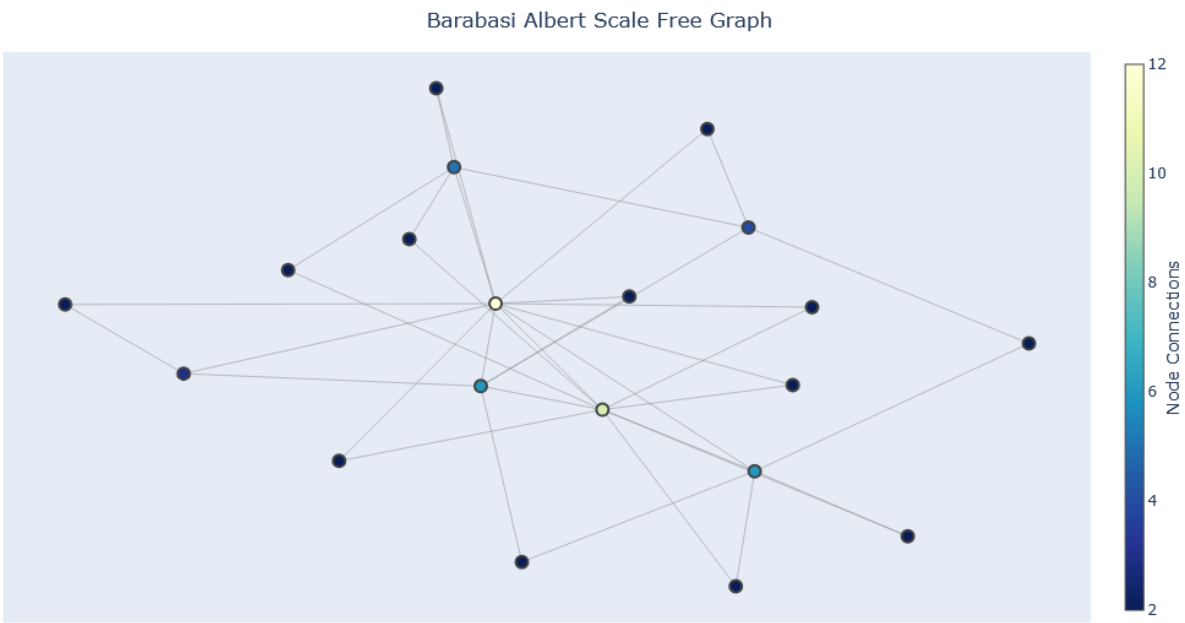


Figure 2.3: Barabasi-Albert graph produced by Networkx where variables $N = 20$ and $m_0 = 2$

Chapter 3

Synchrony and Contagion Spread

3.1 History of Synchrony

Synchronous behaviour describes when actions made from objects happen in time with each other, in other words the actions happen at the same time, therefore the objects are in sync. This behaviour can be seen in a vast number of settings such as the pacemaker cells syncing to make the heart beat[15] and certain species of fireflies syncing their flashes together[5]. This behaviour can happen when systems are coupled together in some way and hence can influence each other. This was first discovered by Christiaan Huygens in 1656. He developed the first pendulum clock[3] to help sailors measure longitude. Latitude was measured using the position of the sun or stars but for longitude a clock was needed. Spring clocks at the time were routinely off by 15 minutes per day as they depended on how tightly the spring was wound[4]. Huygens' pendulum clock was accurate to around 1 minute per day. His plan was to put the two clocks (in case one was damaged) hanging from a heavy mass on the ship so that the rocking of the ship wouldn't affect the movement of the pendulum. When testing out this arrangement at home, he used two pendulum clocks both hung from the same beam. When he set the clocks in motion he found that after some time they would oscillate 180° out of phase with each other. He disturbed one of them and again after some time they went back to the same motion. So they would essentially sync up, just with a constant phase difference. He first thought that they were influencing each other through air currents, but when he put a board between them they would still follow the same sync-up pattern. He eventually realised they were transferring vibrational energy to each other through the beam they were both

hung from. This beam caused them to become coupled, so they forced each other to go to a stabler state. The beam wouldn't move if they were 180° out of phase since the forces on the beam would cancel out, so this was the most stable state they could find.
In 1967 Winfree was studying synchronisation phenomena in biology[10]. He realised if there was a huge population of limit cycle oscillators, as long as they were weakly coupled and nearly identical oscillators you can just represent them using just their phases, since they will stay in their stable limit cycle if not overly disturbed. This is a very powerful tool, as it takes nodes that depend on many variables, and it reduces them down so that we only care about its phase. Through some more work from Kuramoto[20] and further final simplification, we get his model, which is the most widely used for discussing simple synchronisation phenomena recently.

3.2 Kuramoto Model

The Kuramoto model introduced in 1975 provides a mathematical framework for studying synchronisation [20]. By representing oscillators as simple phase variables coupled through sinusoidal interactions, Kuramoto demonstrated that populations exhibit a sharp phase transition. Below a critical coupling strength, oscillators drift incoherently, above it they lock on to a common frequency and phase. This elegant model has since been applied to phenomena ranging from coupled Josephson junctions [16]to opinion formation[17], providing fundamental insights into collective dynamics. Early Kuramoto studies assumed all-to-all coupling where every oscillator influences every other equally. Real systems, however, are embedded in networks with heterogeneous, sparse connectivity. A neuron synapses with thousands of neighbors, not billions. Power plants connect to regional grids, not every generator worldwide. Over the past two decades, researchers have extended Kuramoto's framework to complex networks, revealing that topology profoundly shapes synchronisation thresholds. Network structure determines whether and when collective behaviour emerges.

3.3 Algbebraic Connectivity

For synchronisation, theoretical work has established that spectral properties of the graph Laplacian, particularly the algebraic connectivity λ_2, the second-smallest eigenvalue, determine critical coupling thresholds[19]. Networks with higher λ_2 synchronise more easily because λ_2 quantifies global connectivity which is how difficult it is to partition the network into weakly coupled subgraphs. Hubs increase λ_2 by providing well-connected nodes that bridge disparate regions. Clustering can decrease λ_2 by creating bottlenecks between tightly-knit communities. This spectral perspective predicts that hub-rich scale-free networks should synchronise most easily, while clustered small-world networks should resist.

3.4 Simple vs. Complex Contagion

We will also investigate further collective dynamics beyond synchronisation. Consider the spread of behaviors requiring social reinforcement such as adopting new technologies, participating in social movements, or changing health habits. The spread of these activities can be considered a complex contagion as individuals usually adopt only after multiple exposures, not from a single contact. Unlike diseases spreading via simple contagion (one infected contact suffices), behaviours often require social proof from several peers before adoption. Granovetter's [18]threshold models formalise this where each individual has an adoption threshold which is the fraction of neighbors that must adopt before they follow suit.

3.5 Research Questions

We set out a few key research questions we aimed to address. Can the same network structure optimise both synchronisation and simple/complex contagion spread or do these dynamics impose conflicting architectural demands? What are the key properties that define the performance of a network for each dynamic? We address this question by systematically comparing Kuramoto synchronisation and threshold-based cascades on Erdos-Renyi, Watts-Strogatz, and Barabasi-Albert networks. We measure critical coupling thresholds for synchronisation, test spectral theoretical predictions and quantify

cascade sizes under threshold dynamics.

Chapter 4

Methods

4.1 Network Generation

Erdos-Renyi, Watts-Strogatz and Barabasi-Albert networks were investigated. For our threshold model experiment, each network was generated with 300 nodes. For synchronisation investigation using the Kuramoto model, each was generated with 100 nodes and for the SIR model each network was generated with 500 nodes. In all three cases, all networks were designed to have a mean degree of roughly 6 connections per node. Erdos-Renyi networks were generated by connecting each pair of nodes with probability = 6/(N-1). This creates random graphs with approximately Poisson degree distributions. Watts-Strogatz networks began as regular ring lattices where each node was connected to its three nearest neighbours on either side. We then rewired each edge with probability = 0.1, preserving one endpoint of the edge while randomly selecting the other. this procedure maintains high clustering, while introducing some shortcuts through the network. Barabasi-Albert networks were grown via preferential attachment. Starting with 3 fully connected nodes, further nodes are sequentially added, each assigned connection to $m_0 = 3$ existing nodes with probability proportional to those nodes' current degree. This growth mechanism produces scale-free networks with power-law degree distributions and hub-dominated topologies. The m = 3 parameter was chosen as for a large network the number of edges with then roughly be equal to mN, as each node introduced to the network after the initial three will bring m edges. As each edge connects to nodes, the mean degree will be equal to 2m = 6. All networks were generated using NetworkX, with fixed seeds for reproducibility. We computed structural properties including degree dis-

tribution, average path length, clustering coefficient and the graph Laplacian. The global clustering C was calculated as the average of every node's local clustering coefficient C_i:

$$C = \frac{1}{N} \sum_{i=1}^{N} C_i$$

The Laplacian's eigenvalues were obtained via numpy.linalg.eigenvalsh.

4.2 Kuramoto Model

The Kuramoto model describes coupled phase oscillators on networks:

$$d\theta_i/dt = \omega_i + (K/k_i) \sum_{j=1}^{N} A_{ij} \sin(\theta_j - \theta_i)$$

where θ_i is the phase of oscillator i, ω_i is its natural frequency, k_i its degree, K is the coupling strength, and A_{ij} is the adjacency matrix. We normalise by degree k_i so that each oscillator responds to the average phase of its neighbours. Natural frequencies were drawn from a Gaussian distribution $\omega_i \sim \mathcal{N}(0, \sigma^2)$ with standard deviation = 0.5. Initial phases were uniformly distributed: $\theta_i \sim \mathcal{U}(0, 2\pi)$. We measured synchronisation via the Kuramoto order parameter: r(t) = $\frac{1}{N} \sum_{j=1}^{N} e^{i\theta_j(t)}$ where i denotes the imaginary unit. The order parameter ranges from r = 0 (complete incoherence) to r = 1 (perfect synchronisation). For each network, we scanned coupling strength from 0 to 4. At each K value, we integrated the system using scipy.integrate.odeint with timestep dt = 0.1 for total time T = 150, sufficient for the system to reach a steady state. Critical coupling K_c was defined as the K value where the r first exceeds 0.5, identifying the onset of synchronisation. We computed the algebraic connectivity λ_2 for each network and compared K_c values against the predicted inversely proportional relationship.

4.3 Threshold Model

The threshold model represents complex contagion where nodes can require multiple exposures before adopting. Node i adopts if $\frac{\sum_j A_{ij} x_j}{k_i} > \tau$ where $x_j \in (0, 1)$ indicates whether neighbour j has adopted and τ is the adoption threshold. We set $\tau = 0.25$, meaning nodes adopt when at least 25% of their neighbours have adopted. Each simu-

lation began by randomly selecting 5 nodes as initial adopters. At each timestep then, all nodes simultaneously evaluated their adoption criteria. Nodes crossing the threshold adopted irreversibly. Iterations continued until no new adoptions occurred or 100 timesteps elapsed. We measured cascade size as the fraction of nodes that eventually adopted. For each network, we averaged cascade sizes over 50 independent trials with different randomly selected initial adopters.

4.4 SIR Model

The Susceptible, Infected and Recovered (SIR) model[13] is one of the simplest ways to model epidemic spread in a given population. The susceptiple people are those who can get infected by the disease/virus, the Infected are those who are infected with it, and the recovered are the individuals who have recovered from it and are, in this model, immune for life. In this simple model, almost everyone in the population starts out as susceptible and some individuals are infected I_0. The Infection then spreads using the following differential equations

$$\frac{dS}{dt} = -\beta SI$$

$$\frac{dI}{dt} = \beta SI - \gamma I$$

$$\frac{dR}{dt} = \gamma I$$

where β is the rate of infection, γ is the rate of recovery and S, I, R are variables indication the number of people (nodes) susceptible, infected and recovered respectively.

In each of the network models in the simulation, an infected node can only infect a node it's connected to. So at each time step, all infected nodes infect their neighbours with a probability based on β. The more infected neighbours a node has, the higher chance it has of becoming infected. $\gamma = \frac{1}{12.4}$ in units of inverse time, as 12.4 days was the average time people spend infected with Covid-19[21]. $\beta = 0.3$ was chosen so that $R_0 = \frac{\beta}{\gamma} > 1$ so that the epidemic can spread. If the ratio $R_0 < 1$ the epidemic cannot spread[13]. The population was chosen $N = 500$ with $I_0 = 5$ nodes initially infected, there were 500 monte carlo runs simulated and the average and standard deviation of those was taken[14]. This was done for each of the three graphs.

Chapter 5

Results

5.1 Network Characteristics

5.1.1 Basic Properties

An Erdos-Renyi, Watts-Strogatz and Barabasi-Albert network was simulated with 500 nodes and average degree of roughly 6 for each to examine each network's key characteristic properties. We see the degree range is much larger for the scale-free network, and much smaller for the small-world network, with the random graph in between. This is as expected with the hub effect and heterogeneous degree distribution for the scale-free network, while with only a random rewiring probability of 0.1 for the small-world network, each node's degree remains close to the mean. Our large clustering value for the small-world and negligible for the random graph is as expected, as the clustering value for a random graph would be expected to approach zero as N approaches infinity. The small-world shows the largest path lengths due to limited shortcuts introduced by rewiring, while the shortest path lengths occur in the scale-free network, as nodes can be connected through hubs easily.

5.1.2 Degree Distributions

Degree distribution plots were also generated on each network with 300 nodes with average degree 6. The random graph conforms to a roughly Poisson degree distribution, while the small-world has a narrower distribution more concentrated near the mean. The scale-free network plotted on a log-log scale shows a roughly straight downward slope due to its

```
ER(n=500, k=6.0)
============================================================
Nodes: 500
Edges: 1489
Average degree: 5.96 ± 2.48
Degree range: [1, 16]
Clustering coefficient: 0.0111
Average path length: 3.68
Diameter: 7

WS(n=500, k=6, β=0.1)
============================================================
Nodes: 500
Edges: 1500
Average degree: 6.00 ± 0.70
Degree range: [4, 9]
Clustering coefficient: 0.4645
Average path length: 5.70
Diameter: 11

BA(n=500, m=3)
============================================================
Nodes: 500
Edges: 1491
Average degree: 5.96 ± 6.85
Degree range: [3, 66]
Clustering coefficient: 0.0559
Average path length: 3.19
Diameter: 5
```

Figure 5.1: Network Properties

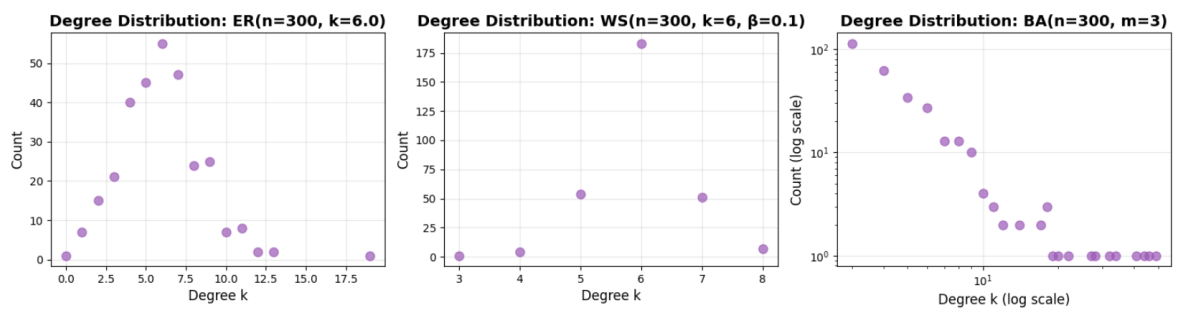


Figure 5.2: Degree Distributions

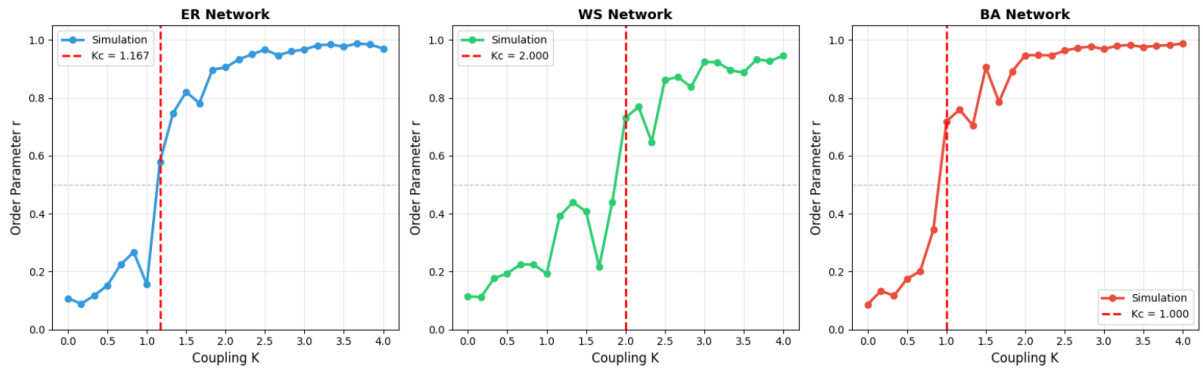


Figure 5.3: Phase Transitions

power-law degree distribution with a constant exponent.

5.1.3 Algebraic Connectivity

A network size of 100 with average degree 6 was chosen for the upcoming experiments on Kuramoto synchronisation. The second smallest eigenvalue of the graph Laplacian for each network was calculated i.e. the algebraic connectivity. For the scale-free network, this value was 1.412, random graph : 0.713, small-world : 0.364.

5.2 Kuramoto Synchronisation

All three networks exhibit sharp transitions from incoherent to synchronised as coupling increases. Critical couplings found: Scale-Free: $K_c = 1$ Random Graph: $K_c = 1.17$ Small-World: $K_c = 2$

The scale-free network requires the smallest critical coupling value. The hubs in this network average many neighbours, so computing the average of all its neighbours provides

Spectral Prediction: $K_c \propto 1/\lambda_2$

Network	λ_2	Kc	Trend
BA	1.41	1.00	High λ_2 → Low Kc
ER	0.71	1.17	Medium
WS	0.36	2.00	Low λ_2 → High Kc

Figure 5.4: Spectral Verification

a stable average and the hub becomes a stable reference point for the network's collective state. These networks also have the shortest paths as most nodes are quite close to the hub, so information about the global synchronisation state propagates very quickly. Hubs increase the λ_2 value as to partition a Barabasi-Albert network you'd need to cut through hubs removing many edges. The small-world network required the largest coupling value to synchronise. This network contains many local clusters which can all synchronise to different phases while there's limited inter cluster links. We see a smaller λ_2 value as this network is easy to partition with poor global connectivity that requires a greater coupling value to overcome.

Our findings are consistent with the theory that the critical coupling value will be inversely proportional to the λ_2. We see the largest λ_2 value for the scale-free network which has the smallest critical coupling and vice versa for the small-world network, with the random graph in between for both.

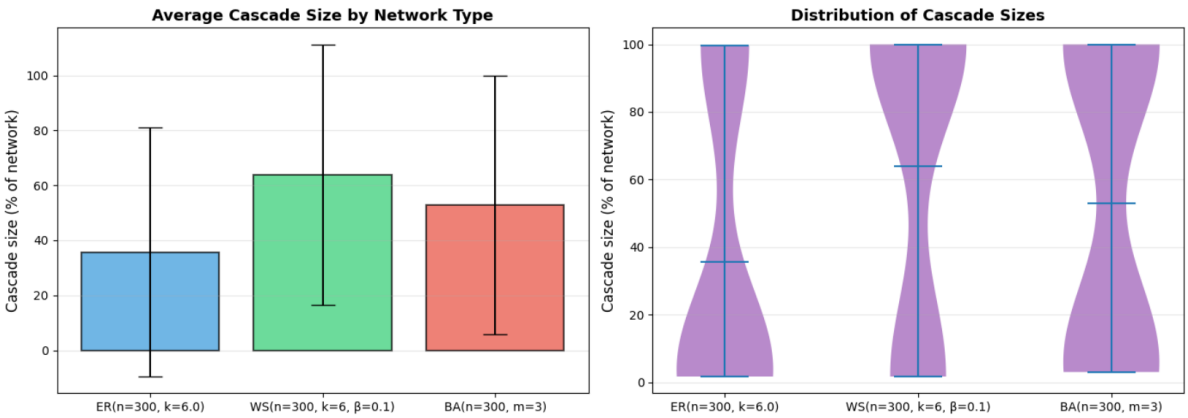


Figure 5.5: Cascade Sizes

5.3 Complex Contagion

5.3.1 Threshold Cascades

Mean cascade sizes (+- std): WS: 64% +- 47% BA: 53% +- 47% ER: 36% +- 45%

The small-world network, which performed worst for synchronisation, produces the largest cascade sizes here. The scale-free network which synchronised most easily shows moderate cascade performance. Large standard deviations represent the all-or nothing dynamics evident in the results. Most cascades either die out early or propagate the network. The initial seed fraction of 5 nodes initially adopted out of 300 appears right at the critical threshold.

5.3.2 The Role of Clustering

The different results from the two experiments stem from different key network properties that determine performance. Synchronisation requires global coordination where all oscillators must align through network paths. This favours high λ_2 (global connectivity) provided by hub structures. Complex contagion requires local reinforcement as nodes need multiple neighbours adopting to cross a threshold. In small-world networks high clustering creates tightly-knit groups where friends know each other. When one member adopts from a friend, mutual friends see multiple adopters, quickly crossing the 25% threshold. Cascades spread through these reinforcing triangles. Local groups adopt in unison, creating avalanches that percolate through the network. In BA networks, hubs

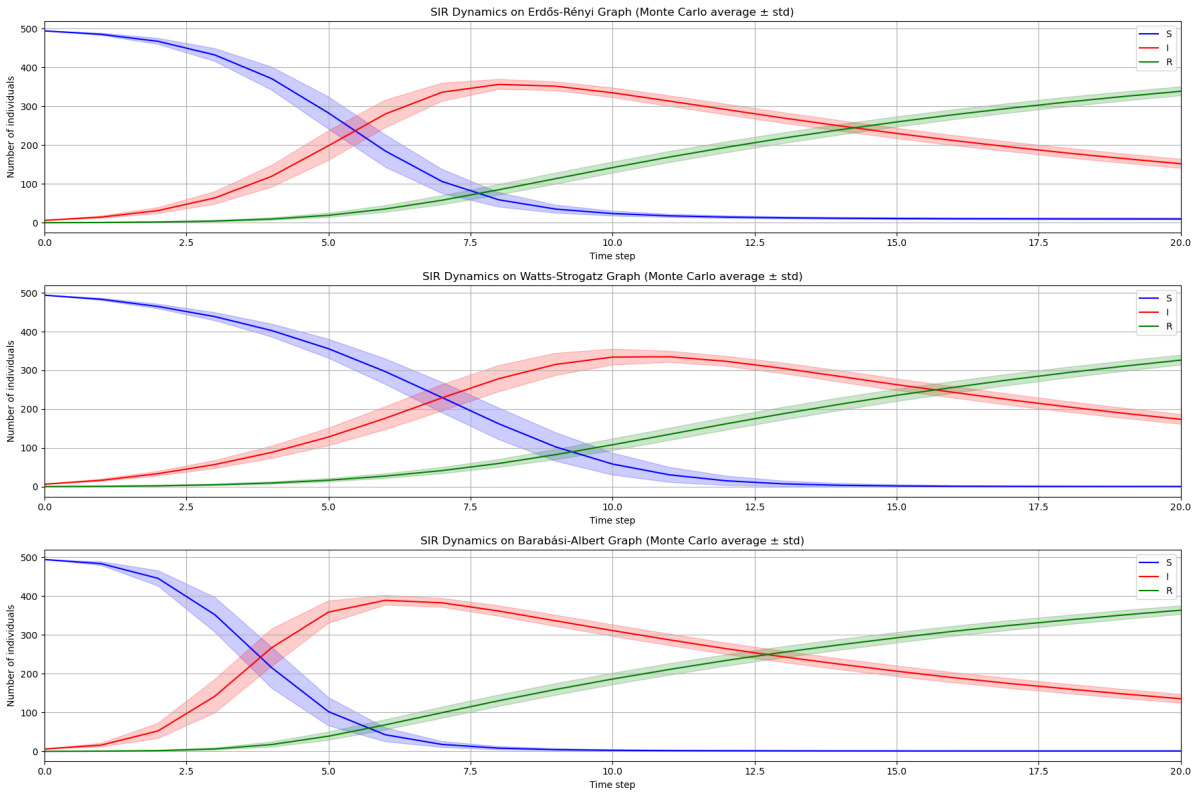
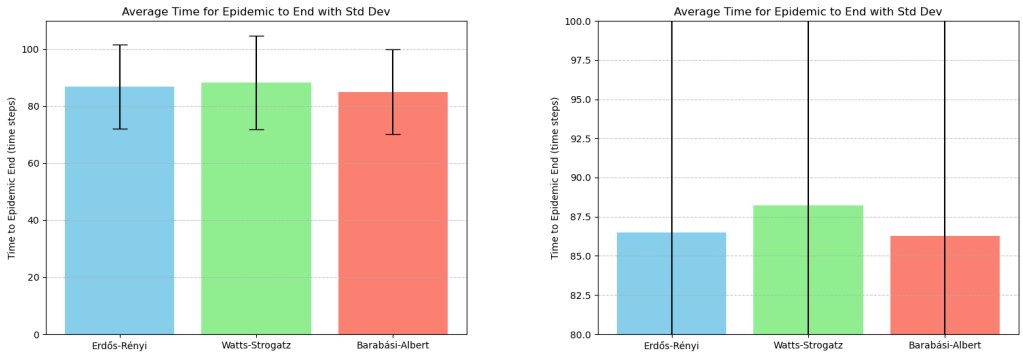


Figure 5.6: Results from the SIR model monte carlo simulation. The plots show the number of nodes that are susceptible (blue), infected (red) and recovered (green) with each plot being set on a different network(ER top, WS middle, BA bottom)

lack clustering around their neighbours. Even if hub with 40 connections adopts, those 40 neighbours typically don't connect to each other. Each neighbours sees only the hub adopting, a single exposure. For a neighbour with 5 total connections, this represents just 20% adoption, below the 25% threshold. Cascades stall despite hub presence because complex contagion requires local reinforcement, not just reach.

5.4 Simple Contagion - SIR

Looking at Figure 5.6 all three models have a very similar shape demonstrating how, no matter the graph, the final result will be the same, just with some difference in the time taken to get there. It's easily seen that the Barabasi-Albert graph has the steepest red (infected) and blue (susceptible) curves, while the Watts-Strogatz graph has the gentlest slopes. This indicates that an epidemic spreads better on a Barabasi Albert network then on either of the others. Looking at the time it takes to become recovered, we can see in



(a) (b)

Figure 5.7: Bar charts describing how long it takes for every node in the network to become recovered. **(b)** is a zoomed in version of **(a)**

Figure 5.7 that BA takes the shortest and WS takes the longest amount of time.

So just like in the Kuramoto model, epidemic spread prefers hubs to clusters. Once a hub is infected, a significant number of the population will become infected since the hub has such a high degree, this then means more hubs will become infected and those hubs will reach more nodes, so there's a cascading effect caused by the hubs and they act like superspreaders. In contrast, the clusters cause the communities to become infected locally, but since there's only a relatively small number of shortcuts between communities, so the infection grows slower. In the ER network degree is random and uniform (poisson degree distribution) so disease spreads evenly through the network. These results again show the effect that network type has on the dynamics of the system as a whole.

Chapter 6

Conclusion

We looked at 3 different models of networks the ER, WS and BA. The dynamics of the Kuramoto model preferred the BA network, that is, the oscillators synced up easier to that model than the others because the hubs acted like trend-setters to their connected nodes. This increased the algebraic connectivity (λ_2) as the hubs make it hard to partition the graph. The WS network exhibited the lowest K_c and the lowest λ_2 because the local clusters make global synchronisation difficult and the minimal 'shortcuts' make the global connectivity lower. This is similar to the results found in the SIR model. The BA had hubs acting like superspreaders while the community structure of the WS caused slower infection. In contrast the complex contagion showed the strongest cascading in the WS because of its community structure, whereas the BA network had low cascading because there was little community structure.

Overall, these results demonstrate that no single network structure optimises all dynamical processes. Global coordination processes, such as synchronisation and epidemic spreading, benefit from hub-dominated, highly connected networks, whereas processes requiring reinforcement, such as complex contagion, depend on clustered local structure. This highlights a fundamental trade-off in network design, with important implications for understanding and engineering real-world systems.

Appendix A

Code

A.1 Python-Fergal

Code used to generate Figure 5.6

```
import networkx as nx
import numpy as np
import matplotlib.pyplot as plt

beta = 0.3       # infection rate
gamma = 1/12.39 # recovery rate
population = 500
initial_infected = 6
timesteps = 25
monte_carlo_runs = 500  #number of independent runs

def create_graphs(n):
    p = 0.012
    G_er = nx.erdos_renyi_graph(n, p)
    k = 6
    rewiring_p = 0.1
    G_ws = nx.watts_strogatz_graph(n, k, rewiring_p)
    m = 3
    G_ba = nx.barabasi_albert_graph(n, m)
```

```python
    return G_er, G_ws, G_ba

def initialize_states(G):
    states = {}
    for node in G.nodes:
        states[node] = 'S'
    infected_nodes = np.random.choice(list(G.nodes), initial_infected, replace=False)
    for node in infected_nodes:
        states[node] = 'I'
    return states

def step_SIR(G, states):
    new_states = states.copy()
    for node in G.nodes:
        if states[node] == 'S':
            neighbors = list(G.neighbors(node))
            infected_neighbors = sum(states[n] == 'I' for n in neighbors)
            p_infection = 1 - (1 - beta) ** infected_neighbors
            if np.random.rand() < p_infection:
                new_states[node] = 'I'
        elif states[node] == 'I':
            if np.random.rand() < gamma:
                new_states[node] = 'R'
    return new_states

def run_simulation(G):
    states = initialize_states(G)
    history = {'S': np.zeros(timesteps), 'I': np.zeros(timesteps), 'R': np.zeros(timesteps)}
    for t in range(timesteps):
        counts = {'S': 0, 'I': 0, 'R': 0}
        for s in states.values():
            counts[s] += 1
```

```
        for key in counts:
            history[key][t] = counts[key]
        states = step_SIR(G, states)
    return history

def monte_carlo_simulation(G, runs=monte_carlo_runs):
    all_runs = {'S': [], 'I': [], 'R': []}
    for _ in range(runs):
        history = run_simulation(G)
        for c in all_runs:
            all_runs[c].append(history[c])
    for c in all_runs:
        all_runs[c] = np.array(all_runs[c])
    mean = {c: np.mean(all_runs[c], axis=0) for c in all_runs}
    std = {c: np.std(all_runs[c], axis=0) for c in all_runs}
    return mean, std

def plot_monte_carlo_results(means, stds, graph_names):
    plt.figure(figsize=(18, 12))
    compartments = ['S', 'I', 'R']
    colors = {'S': 'blue', 'I': 'red', 'R': 'green'}
    for i, graph_name in enumerate(graph_names):
        plt.subplot(3, 1, i+1)
        for c in compartments:
            mean_curve = means[i][c]
            std_curve = stds[i][c]
            plt.plot(mean_curve, label=c, color=colors[c])
            plt.fill_between(range(timesteps), mean_curve - std_curve, mean_curve + std_curve, color=colors[c], alpha=0.2)
        plt.title(f'SIR Dynamics on {graph_name} Graph (Monte Carlo average ± std)')
        plt.xlabel('Time step')
        plt.ylabel('Number of individuals')
        plt.xlim(0,20)
```

```
        plt.legend()
        plt.grid(True)
    plt.tight_layout()
    plt.savefig('SIRresults3')
    plt.show()
if __name__ == "__main__":
    n = population
    G_er, G_ws, G_ba = create_graphs(n)
    means_er, stds_er = monte_carlo_simulation(G_er)
    means_ws, stds_ws = monte_carlo_simulation(G_ws)
    means_ba, stds_ba = monte_carlo_simulation(G_ba)
    plot_monte_carlo_results([means_er, means_ws, means_ba], [stds_er,
    ↪ stds_ws, stds_ba], ['Erdős-Rényi', 'Watts-Strogatz',
    ↪ 'Barabási-Albert'])
```

Code used to generate Figure 5.7

```
import networkx as nx
import numpy as np
import matplotlib.pyplot as plt

beta = 0.3
gamma = 1/12.4
population = 500
initial_infected = 6
timesteps = 200  # extended so epidemic ends
monte_carlo_runs = 500

def create_graphs(n):
    p = 0.012
    G_er = nx.erdos_renyi_graph(n, p)
    k = 6
    rewiring_p = 0.1
    G_ws = nx.watts_strogatz_graph(n, k, rewiring_p)
```

```python
    m = 3
    G_ba = nx.barabasi_albert_graph(n, m)
    return G_er, G_ws, G_ba

def initialize_states(G):
    states = {node: 'S' for node in G.nodes}
    infected_nodes = np.random.choice(list(G.nodes), initial_infected, replace=False)
    for node in infected_nodes:
        states[node] = 'I'
    return states

def step_SIR(G, states):
    new_states = states.copy()
    for node in G.nodes:
        if states[node] == 'S':
            infected_neighbors = sum(states[n] == 'I' for n in G.neighbors(node))
            p_infection = 1 - (1 - beta) ** infected_neighbors
            if np.random.rand() < p_infection:
                new_states[node] = 'I'
        elif states[node] == 'I':
            if np.random.rand() < gamma:
                new_states[node] = 'R'
    return new_states

def run_simulation(G):
    states = initialize_states(G)
    for t in range(timesteps):
        # Count infectious
        num_infectious = sum(1 for s in states.values() if s == 'I')
        if num_infectious == 0:
            return t  # epidemic ended at time t
        states = step_SIR(G, states)
```

```python
    return timesteps  # epidemic still ongoing at max time

def monte_carlo_duration(G, runs=monte_carlo_runs):
    durations = []
    for _ in range(runs):
        duration = run_simulation(G)
        durations.append(duration)
    durations = np.array(durations)
    mean_duration = np.mean(durations)
    std_duration = np.std(durations)
    return mean_duration, std_duration, durations

def plot_duration_bar(means, stds, graph_names):
    plt.figure(figsize=(8,6))
    x_pos = np.arange(len(graph_names))
    plt.bar(x_pos, means, yerr=stds, capsize=8, color=['skyblue', 'lightgreen', 'salmon'])
    plt.xticks(x_pos, graph_names)
    plt.ylabel('Time to Epidemic End (time steps)')
    plt.title('Average Time for Epidemic to End with Std Dev')
    plt.grid(axis='y', linestyle='--', alpha=0.7)
    plt.savefig('barchartSIR2')
    plt.show()

if __name__ == "__main__":
    n = population
    G_er, G_ws, G_ba = create_graphs(n)
    mean_er, std_er, _ = monte_carlo_duration(G_er)
    mean_ws, std_ws, _ = monte_carlo_duration(G_ws)
    mean_ba, std_ba, _ = monte_carlo_duration(G_ba)
    plot_duration_bar([mean_er, mean_ws, mean_ba], [std_er, std_ws, std_ba], ['Erdős-Rényi', 'Watts-Strogatz', 'Barabási-Albert'])
```

A.2 Python-Kevin

```python
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt
from collections import Counter

# Set random seed for reproducibility
np.random.seed(42)

# Configure matplotlib
plt.rcParams['figure.dpi'] = 100
plt.rcParams['savefig.dpi'] = 150
plt.rcParams['font.size'] = 10


def generate_erdos_renyi(n, avg_degree, seed=None):
    """Generate an Erdős{Rényi random graph."""
    p = avg_degree / (n - 1)
    G = nx.erdos_renyi_graph(n, p, seed=seed)
    G.name = f"ER(n={n}, k={avg_degree:.1f})"
    return G


def generate_watts_strogatz(n, k, beta, seed=None):
    """Generate a Watts{Strogatz small-world graph."""
    G = nx.watts_strogatz_graph(n, k, beta, seed=seed)
    G.name = f"WS(n={n}, k={k}, ={beta})"
    return G


def generate_barabasi_albert(n, m, seed=None):
    """Generate a Barabási{Albert scale-free graph."""
```

```python
    G = nx.barabasi_albert_graph(n, m, seed=seed)
    G.name = f"BA(n={n}, m={m})"
    return G


def network_statistics(G):
    """Compute basic statistics for a network."""
    degrees = dict(G.degree())
    degree_sequence = list(degrees.values())

    stats = {
        'name': G.name if hasattr(G, 'name') else 'Unnamed',
        'n_nodes': G.number_of_nodes(),
        'n_edges': G.number_of_edges(),
        'avg_degree': np.mean(degree_sequence),
        'std_degree': np.std(degree_sequence),
        'max_degree': max(degree_sequence),
        'min_degree': min(degree_sequence),
        'clustering': nx.average_clustering(G),
        'is_connected': nx.is_connected(G),
    }

    if nx.is_connected(G):
        stats['avg_path_length'] = nx.average_shortest_path_length(G)
        stats['diameter'] = nx.diameter(G)
    else:
        largest_cc = max(nx.connected_components(G), key=len)
        G_cc = G.subgraph(largest_cc).copy()
        stats['avg_path_length'] = nx.average_shortest_path_length(G_cc)
        stats['diameter'] = nx.diameter(G_cc)
        stats['largest_cc_fraction'] = len(largest_cc) /
        ↪  G.number_of_nodes()

    return stats
```

```python


def print_network_stats(G):
    """Pretty-print network statistics."""
    stats = network_statistics(G)

    print(f"\n{stats['name']}")
    print("="*60)
    print(f"Nodes: {stats['n_nodes']}")
    print(f"Edges: {stats['n_edges']}")
    print(f"Average degree: {stats['avg_degree']:.2f} ± {stats['std_degree']:.2f}")
    print(f"Degree range: [{stats['min_degree']}, {stats['max_degree']}]")
    print(f"Clustering coefficient: {stats['clustering']:.4f}")
    print(f"Average path length: {stats['avg_path_length']:.2f}")
    print(f"Diameter: {stats['diameter']}")
    if not stats['is_connected']:
        print(f"Largest connected component: {stats['largest_cc_fraction']:.2%}")


n = 500
avg_k = 6

G_er = generate_erdos_renyi(n, avg_k, seed=42)
G_ws = generate_watts_strogatz(n, k=6, beta=0.1, seed=42)
G_ba = generate_barabasi_albert(n, m=3, seed=42)

print_network_stats(G_er)
print_network_stats(G_ws)
print_network_stats(G_ba)


class ThresholdModel:
```

```python
    """
    Threshold model: complex contagion where nodes adopt if enough neighbors have adopted.
    """

    def __init__(self, G, thresholds=None, seed_nodes=None, seed=None):
        self.G = G
        self.n_nodes = G.number_of_nodes()
        self.adj_list = {i: list(G.neighbors(i)) for i in G.nodes()}

        if seed is not None:
            np.random.seed(seed)

        # Initialise thresholds
        if thresholds is None or thresholds == 'uniform':
            self.thresholds = {i: np.random.random() for i in G.nodes()}
        elif isinstance(thresholds, dict):
            self.thresholds = thresholds.copy()
        elif isinstance(thresholds, float):
            self.thresholds = {i: thresholds for i in G.nodes()}

        # Initialise adoption state
        self.adopted = {i: False for i in G.nodes()}

        # Seed initial adopters
        if seed_nodes is not None:
            if isinstance(seed_nodes, int):
                nodes_to_seed = np.random.choice(list(G.nodes()), seed_nodes, replace=False)
            else:
                nodes_to_seed = seed_nodes

            for node in nodes_to_seed:
                self.adopted[node] = True
```

```python

        self.time = 0
        self.history = [sum(self.adopted.values())]

    def step(self):
        """Perform one time step."""
        new_adopted = self.adopted.copy()

        for i in self.G.nodes():
            if not self.adopted[i]:
                neighbors = self.adj_list[i]
                if len(neighbors) > 0:
                    frac_adopted = sum(self.adopted[j] for j in neighbors) / len(neighbors)

                    if frac_adopted >= self.thresholds[i]:
                        new_adopted[i] = True

        self.adopted = new_adopted
        self.time += 1
        self.history.append(sum(self.adopted.values()))

    def run(self, max_steps=100):
        """Run simulation until convergence or max_steps."""
        prev_adopted = -1

        for _ in range(max_steps):
            current_adopted = sum(self.adopted.values())

            if current_adopted == prev_adopted:
                break

            prev_adopted = current_adopted
            self.step()
```

```python

        return sum(self.adopted.values())

    def get_cascade_size(self):
        """Return current number of adopters."""
        return sum(self.adopted.values())

    def get_cascade_fraction(self):
        """Return fraction of nodes that have adopted."""
        return self.get_cascade_size() / self.n_nodes


def plot_cascade_size(history, ax=None, title="Cascade Growth"):
    """Plot cascade size over time."""
    if ax is None:
        fig, ax = plt.subplots(figsize=(10, 5))

    times = np.arange(len(history))

    ax.plot(times, history, linewidth=2.5, color='green', marker='o', markersize=4)
    ax.set_xlabel('Time step', fontsize=12)
    ax.set_ylabel('Number of adopters', fontsize=12)
    ax.set_title(title, fontsize=14, fontweight='bold')
    ax.grid(alpha=0.3)

    return ax


def plot_degree_distribution(G, ax=None, log_scale=True):
    """Plot degree distribution."""
    if ax is None:
        fig, ax = plt.subplots(figsize=(8, 6))

```

```python
    degrees = [G.degree(n) for n in G.nodes()]
    degree_counts = np.bincount(degrees)
    degree_values = np.arange(len(degree_counts))

    mask = degree_counts > 0
    x = degree_values[mask]
    y = degree_counts[mask]

    if log_scale:
        ax.loglog(x, y, 'o', markersize=8, alpha=0.7, color='purple')
        ax.set_xlabel('Degree k (log scale)', fontsize=12)
        ax.set_ylabel('Count (log scale)', fontsize=12)
    else:
        ax.plot(x, y, 'o', markersize=8, alpha=0.7, color='purple')
        ax.set_xlabel('Degree k', fontsize=12)
        ax.set_ylabel('Count', fontsize=12)

    ax.set_title(f'Degree Distribution: {G.name}', fontsize=14, fontweight='bold')
    ax.grid(alpha=0.3)

    return ax


n = 300
n_trials = 30
threshold_value = 0.25
n_seeds = 5

networks = [G_er, G_ws, G_ba]
results_exp2 = {G.name: [] for G in networks}

for G in networks:
    print(f"Running {n_trials} trials on {G.name}...")
```

```python
    for trial in range(n_trials):
        tm = ThresholdModel(G, thresholds=threshold_value,
                            seed_nodes=n_seeds, seed=trial)
        cascade_size = tm.run(max_steps=100)
        results_exp2[G.name].append(cascade_size / n)

# Results
print("\n" + "="*60)
print(f"RESULTS: Cascade Size (threshold={threshold_value}, seeds={n_seeds})")
print("="*60)
for name, fractions in results_exp2.items():
    print(f"{name:30s}: {np.mean(fractions):.2\%} ± {np.std(fractions):.2\%}")


# Visualise experiment
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Bar chart
names = list(results_exp2.keys())
means = [np.mean(results_exp2[name]) * 100 for name in names]
stds = [np.std(results_exp2[name]) * 100 for name in names]

colours = ['blue', 'green', 'red']
axes[0].bar(range(len(names)), means, yerr=stds, color=colours,
            alpha=0.7, capsize=10, edgecolor='black', linewidth=1.5)
axes[0].set_xticks(range(len(names)))
axes[0].set_xticklabels(names, fontsize=10)
axes[0].set_ylabel('Cascade size (\% of network)', fontsize=12)
axes[0].set_title('Average Cascade Size by Network Type', fontsize=13, fontweight='bold')
axes[0].grid(alpha=0.3, axis='y')

```

```python
# Violin plot
data_for_violin = [np.array(results_exp2[name]) * 100 for name in names]
parts = axes[1].violinplot(data_for_violin, positions=range(len(names)),
                           showmeans=True, showextrema=True)

for pc in parts['bodies']:
    pc.set_facecolor('purple')
    pc.set_alpha(0.7)

axes[1].set_xticks(range(len(names)))
axes[1].set_xticklabels(names, fontsize=10)
axes[1].set_ylabel('Cascade size (\% of network)', fontsize=12)
axes[1].set_title('Distribution of Cascade Sizes', fontsize=13,
↪ fontweight='bold')
axes[1].grid(alpha=0.3, axis='y')

plt.tight_layout()
plt.show()



import numpy as np
import matplotlib.pyplot as plt
import networkx as nx
from scipy.integrate import odeint

np.random.seed(42)
plt.rcParams['figure.dpi'] = 100


class Kuramoto:
    """
    Kuramoto model: coupled phase oscillators on a network.

```

```
    Uses degree normalisation (each oscillator responds to
    average of neighbors, not sum).
    """

    def __init__(self, adjacency_matrix, coupling, frequencies=None,
    ↪   dt=0.1):
        """
        Parameters
        ----------
        adjacency_matrix : np.ndarray
            NxN adjacency matrix
        coupling : float
            Coupling strength K
        frequencies : np.ndarray, optional
            Natural frequencies. If None, drawn from N(0, 0.5)
        dt : float
            Integration timestep
        """
        self.adj_matrix = np.array(adjacency_matrix)
        self.N = len(self.adj_matrix)
        self.coupling = coupling
        self.dt = dt

        # Natural frequencies
        if frequencies is None:
            self.frequencies = np.random.normal(0, 0.5, self.N)
        else:
            self.frequencies = np.array(frequencies)

        # Degrees for normalisation
        self.degrees = np.sum(self.adj_matrix, axis=1)
        self.degrees[self.degrees == 0] = 1  # Avoid division by zero

        # Initialize phases randomly
```

```python
        self.phases = np.random.uniform(0, 2*np.pi, self.N)

    def derivative(self, phases, t):
        """
        Compute d/dt.

        Normalise by degree so each node responds to AVERAGE of neighbors.
        """
        # Phase differences:
        phase_diffs = phases[np.newaxis, :] - phases[:, np.newaxis]

        # Coupling term:
        coupling_sum = np.sum(self.adj_matrix * np.sin(phase_diffs), axis=1)

        dphases_dt = self.frequencies + self.coupling * coupling_sum / self.degrees

        return dphases_dt

    def integrate(self, T, initial_phases=None):
        """
        Integrate the system for time T.

        Parameters
        ----------
        T : float
            Total integration time
        initial_phases : np.ndarray, optional
            Starting phases. If None, use random.

        Returns
        -------
        t : np.ndarray
```

```
            Time points
        phases : np.ndarray
            Phases at each time point (shape: [len(t), N])
        order_params : np.ndarray
            Order parameter at each time point
        """
        if initial_phases is not None:
            self.phases = initial_phases
        else:
            self.phases = np.random.uniform(0, 2*np.pi, self.N)

        # Time points
        t = np.arange(0, T, self.dt)

        # Integrate
        phases = odeint(self.derivative, self.phases, t)

        # Compute order parameter at each time
        order_params = np.abs(np.mean(np.exp(1j * phases), axis=1))

        # Store final state
        self.phases = phases[-1]

        return t, phases, order_params

    def order_parameter(self, phases=None):
        """Compute order parameter r = |⟨e^(iθ)⟩|."""
        if phases is None:
            phases = self.phases
        return np.abs(np.mean(np.exp(1j * phases)))


def find_critical_coupling(adj_matrix, K_range=(0, 4), n_points=20,
↪   T_integrate=150):
```

```python
    """
    Find critical coupling by scanning K values.

    Returns
    -------
    K_values : np.ndarray
        Coupling values tested
    r_steady : np.ndarray
        Steady-state order parameter for each K
    Kc : float
        Critical coupling (where r crosses 0.5)
    """
    K_values = np.linspace(K_range[0], K_range[1], n_points)
    r_steady = []

    for K in K_values:
        model = Kuramoto(adj_matrix, coupling=K, dt=0.1)
        t, phases, r = model.integrate(T=T_integrate)

        # Take mean of last 20\% as steady state
        r_steady.append(np.mean(r[int(0.8*len(r)):]))

    r_steady = np.array(r_steady)

    # Find where r crosses 0.5
    idx = np.where(r_steady >= 0.5)[0]
    if len(idx) > 0:
        Kc = K_values[idx[0]]
    else:
        Kc = None

    return K_values, r_steady, Kc


```

```python
# Generate networks
n = 100  # Smaller for faster computation

G_er = nx.erdos_renyi_graph(n, 6/(n-1), seed=42)
G_ws = nx.watts_strogatz_graph(n, 6, 0.1, seed=42)
G_ba = nx.barabasi_albert_graph(n, 3, seed=42)

networks = {
    'ER': G_er,
    'WS': G_ws,
    'BA': G_ba
}


# Find critical coupling for each
results = {}

for name, G in networks.items():
    adj = nx.adjacency_matrix(G).toarray()
    K_vals, r_vals, Kc = find_critical_coupling(adj, K_range=(0, 4),
    ↪  n_points=25)
    results[name] = {'K': K_vals, 'r': r_vals, 'Kc': Kc}

    if Kc is not None:
        print(f"{name}: Kc  {Kc:.3f}")
    else:
        print(f"{name}: No sync in range")


# Plot all three
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
colours = {'ER': 'blue', 'WS': 'green', 'BA': 'red'}

for ax, (name, data) in zip(axes, results.items()):
```

```python
    ax.plot(data['K'], data['r'], 'o-', linewidth=2.5, markersize=6,
            color=colours[name], label='Simulation')

    if data['Kc'] is not None:
        ax.axvline(data['Kc'], color='red', linestyle='--', linewidth=2,
                   label=f"Kc = {data['Kc']:.3f}")

    ax.axhline(0.5, color='gray', linestyle='--', linewidth=1, alpha=0.5)
    ax.set_xlabel('Coupling K', fontsize=12)
    ax.set_ylabel('Order Parameter r', fontsize=12)
    ax.set_title(f'{name} Network', fontsize=13, fontweight='bold')
    ax.set_ylim([0, 1.05])
    ax.legend(fontsize=10)
    ax.grid(alpha=0.3)

plt.tight_layout()
plt.show()


# Calculate spectral properties

spectral_results = {}

for name, G in networks.items():
    # Compute graph Laplacian
    L = nx.laplacian_matrix(G).toarray()

    # Get eigenvalues
    eigenvalues = np.linalg.eigvalsh(L)
    eigenvalues = np.sort(eigenvalues)

    # second-smallest (first is always 0)
    lambda2 = eigenvalues[1]

```

```
    Kc_theory = 1.0 / lambda2
    Kc_measured = results[name]['Kc']

    spectral_results[name] = {
        'lambda2': lambda2,
        'Kc_theory': Kc_theory,
        'Kc_measured': Kc_measured
    }
```

Bibliography

[1] P. Erdos & A. Reyni *On Random Graphs*, Publicationes Mathematicae Debrecen (1958)

[2] Albert-Laszlo Barabasi and Reka Albert *Emergence of Scaling in Random Networks*, American Association for the Advancement of Science (1999).

[3] Christiaan Huygens *Horologium Oscillatorium*, apud F. Muguet (1673)

[4] Charles W. Rogers *Huygens's Pendulum Clocks*, EBSCO (2022)

[5] Devin Reese *The Secret of Fireflies' Synchronous Flashing*, Nautilus (2026)

[6] Stanley Milgram *The Individual In a Social World, 3rd Edition*, **322-336**, Printer & Martin Ltd (2010)

[7] Manfred Kochen, Ithiel de Sola Pool *Contacts and Influence*, Elsevie Sequoia (1978/79)

[8] Frigyes Karinthy *Chains*, (1929)

[9] Steven Strogatz & Duncan Strogatz *Collective dynamics of 'small-world' networks*, Nature **393** 440 (June 1998)

[10] A. Winfree *Biological rhythms and the behavior of populations of coupled oscillators*, Journal of Theoretical Biology (1967)

[11] Yoshiki Kuramoto, H. Araki *Lecture Notes in Physics, International Symposium on Mathematical Problems in Theoretical Physics*, Vol. 39. Springer-Verlag, New York. p. 420 (1975)

[12] Networkx Library.

[13] Ross Beckley, Cametria Weatherspoon, Michael Alexander, Marissa Chandler , Anthony Johnson, and Ghan S Bhatt *Modeling epidemics with differential equations*, Tennessee State University (2013)

[14] W. K. Hastings *Monte Carlo Sampling Methods Using Markov Chains and Their Applications*, Biometrika (1970)

[15] D C Michaels 1, E P Matyas, J Jalife *Mechanisms of sinoatrial pacemaker synchronization: a new hypothesis*, Department of Pharmacology, SUNY/Health Science Center, Syracuse (1987)

[16] Synchronization of Josephson junctions in seriesarray, Abhijit Bhattacharyya (2023)

[17] Ziqiao Zhang, Said Al-Abri, Fumin Zhang, A generalized Kuramoto model for opinion dynamics on the unit sphere, Automatica, Volume 171, 2025, 111957, ISSN 0005-1098, https://doi.org/10.1016/j.automatica.2024.111957. (https://www.sciencedirect.com/science/article/pii/S0005109824004515)

[18] Threshold Models of Collective Behavior, Mark Granovetter (1978)

[19] Onset of synchronization in large networks of coupled oscillators, Juan G. Restrepo Edward Ott and Brian R. Hunt (2005)

[20] Kuramoto, Y. (1975). Self-entrainment of a population of coupled non-linear oscillators. In: Araki, H. (eds) International Symposium on Mathematical Problems in Theoretical Physics. Lecture Notes in Physics, vol 39. Springer, Berlin, Heidelberg. https://doi.org/10.1007/BFb0013365

[21] Xi He, Eric H. Y. Lau, Peng Wu, Xilong Deng, Jian Wang, Xinxin Hao, Yiu Chung Lau, Jessica Y. Wong, Yujuan Guan, Xinghua Tan, Xiaoneng Mo, Yanqing Chen, Baolin Liao, Weilie Chen, Fengyu Hu, Qing Zhang, Mingqiu Zhong, Yanrong Wu, Lingzhai Zhao, Fuchun Zhang, Benjamin J. Cowling, Fang Li & Gabriel M. Leung *Temporal dynamics in viral shedding and transmissibility of COVID-19*, Nature Medecine (2020)